FACULTY OF SCIENCE AND NATURAL RESOURCES

SV40303

SCIENTIFIC DATA VISUALISATION

SEMESTER 1, 2025/ 2026 SESSION

PREPARED FOR:

PROF. ABDULLAH BADE

ASSIGNMENT 2: Advanced Medical Data Visualizer (DICOM)

PREPARED BY:

NO.	NAME	NO. MATRICS
1.	WARDAH HANAANI BINTI AKHMAL	BS22110420
2.	NUR AINA NADIRAH BINTI KAMARUL AZHAR	BS22110050

TABLE OF CONTENT

Introduction

For Assignment 2, we were tasked to prepare an advanced medical visualizer that handles DICOM (Digital Imaging and Communications in Medical) dataset. Several classes are introduced with functions that help in handling the dataset to be visualize.

DetachableSliceViewer class is the class used to handle the window that shows the slicing of the DICOM dataset loaded into the system. There are a few functions used to show the slicing of the DICOM dataset loaded, such as the functions that update the color changed on any tab functions that are rendered on the 3D image, orientation, transformation, and the slider that shows which part of the slicing is currently shown on the window.

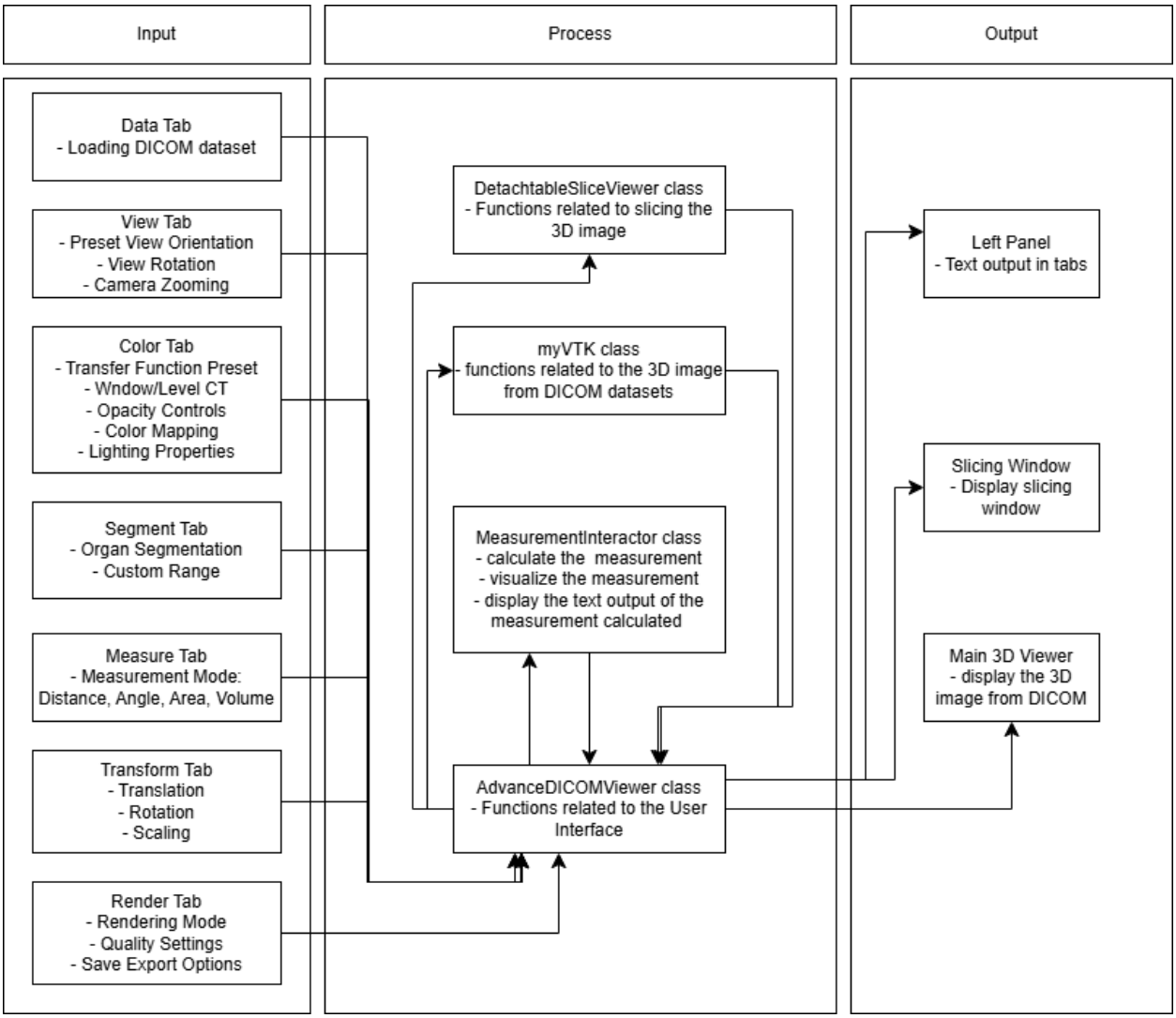
HelpDialog class is used to show a dialog using the QDialog that has the information and direction on how to use or where to click to make changes to the 3D image that will or has been loaded into the system.

Class myVTK is used to render the VTK-related variables, such as the renderer, window, interactor, mapper, actors, and scene, and functions that read the DICOM dataset, set the coordinate from the DICOM loaded with the coordinate, change the transformations such as translation, rotation, and scaling, and other rendering pipelines. There are also functions for camera settings, the slicing camera that shows different orientations of the slicing and information on the slicing to be rendered on the slicing window.

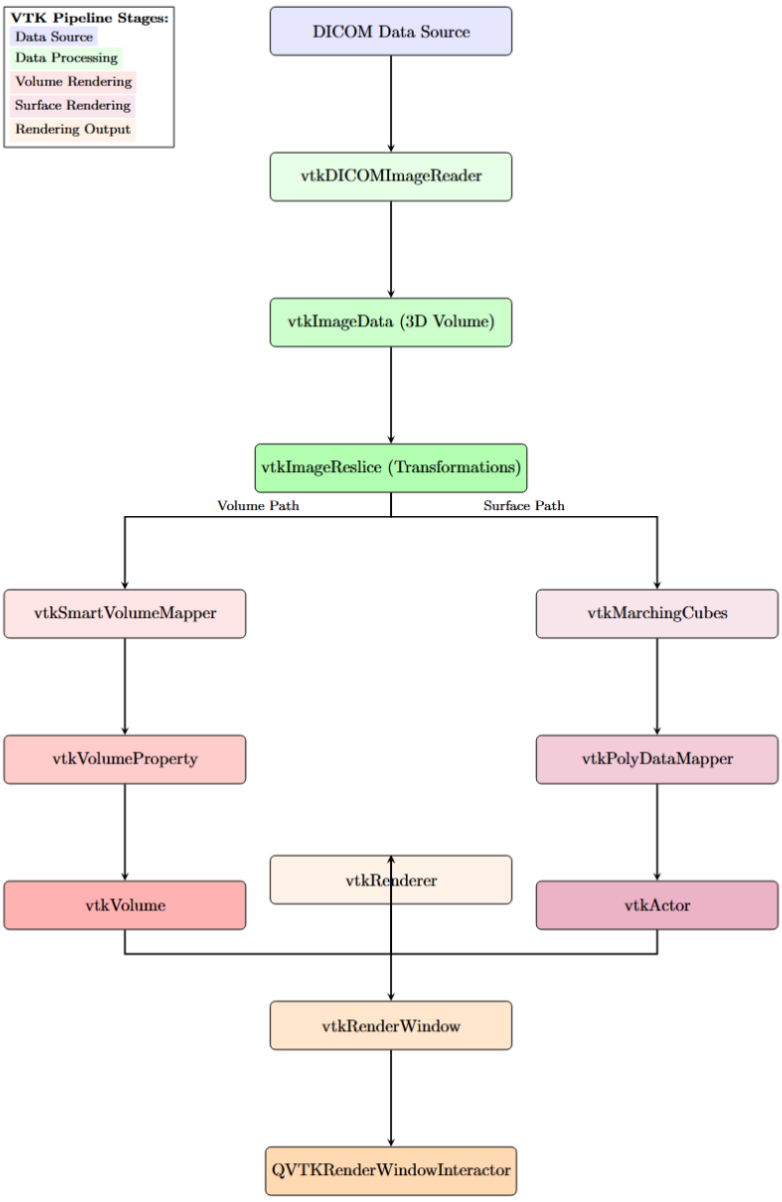
MeasurementInteractor class is used to handle all measurement-related functions. There are several functions that handle the calculations for distance, angle, area, and volume such as _measure_distance, _measure_angle, _measure_area, and _measure_vollume with a few additional functions that help to finalize the points marked for area and volume as they did not have an assigned number of points needed to measure. To visualize the calculations, functions _mark_point, _draw_line, _draw_polygon, and _draw_tetrahedron are used to show the line and points for marking distance between two points and angle between three points, and lines, points, and polygon or tetrahedron drawn to visualize the area and volume. A function named on_left_click is used to control the marking of the points on the 3D image shown using mouse.

The AdvancedDICOMViewer class is where the user interface (UI) is designed by setting the UI design style, creating the left panel and tabs with different functions. Then, there are several functions used to connect the toggles in the tabs and functions that connect the input from the toggles by appending the functions from other classes to make changes to the 3D image.

System Architecture



System Flowchart



Algorithms

Algorithm 1 Main Application Controller

```
CLASS AdvancedDICOMViewer EXTENDS QMainWindow
                                        ▷ MVC Controller coordinating all components

ATTRIBUTES:
    vtk_app: myVTK                                  ▷ 3D rendering engine reference
    loaded_datasets: Dict[str, DatasetInfo]         ▷ Loaded DICOM datasets
    slice_viewer: DetachableSliceViewer             ▷ 2D slice viewer dock
    tabs: QTabWidget                                ▷ 7 functional control tabs
    vtk_widget: QVTKRenderWindowInteractor          ▷ 3D viewport

procedure SETUP_UI
    Create central widget and main layout
    Create splitter(LeftPanel, RightPanel)
    LeftPanel ← create_left_panel()                 ▷ Control tabs
    RightPanel ← create_viewport_panel()            ▷ 3D view
    Setup menus, toolbar, statusbar
end procedure

procedure INITIALIZE_VTK
    vtk_app ← new myVTK()
    vtk_app.start(vtk_widget)                       ▷ Initialize VTK pipeline
end procedure
```

Algorithm 2 VTK Rendering Engine (myVTK)

```
CLASS myVTK
                                        ▷ Core rendering engine managing VTK pipeline

ATTRIBUTES:
    renderer: vtkRenderer                           ▷ 3D scene renderer
    window: vtkRenderWindow                         ▷ Render window
    interactor: vtkRenderWindowInteractor           ▷ User interaction
    imageData: vtkImageData                         ▷ DICOM volume data
    volumeMapper: vtkSmartVolumeMapper              ▷ Volume rendering
    volumeProperty: vtkVolumeProperty               ▷ Visual properties

procedure LOAD_DICOM_DATA(directory)
    reader ← new vtkDICOMImageReader()
    reader.SetDirectoryName(directory)
    reader.Update()
    imageData ← reader.GetOutput()
                                        ▷ Extract metadata: dimensions, spacing, scalar range
    dims ← imageData.GetDimensions()
    spacing ← imageData.GetSpacing()
    scalar_range ← imageData.GetScalarRange()
    return success
end procedure

procedure SETUP_RENDERING_PIPELINE(vtk_widget)
    renderer ← new vtkRenderer()
    window ← vtk_widget.GetRenderWindow()
    window.AddRenderer(renderer)
    interactor ← window.GetInteractor()
    interactor.SetInteractorStyle(vtkInteractorStyleTrackballCamera)
end procedure
```

Algorithm 3 DICOM Data Processing Pipeline

Require: DICOM directory containing slice images
Ensure: 3D rendered volume with interactive controls

```
procedure DICOM Processing Pipeline(directory)
    Phase 1: Data Loading & Reconstruction
        Read DICOM slices → vtkDICOMImageReader
        Stack slices → vtkImageData (3D volume)
        Extract metadata → dimensions, spacing, HU range

    Phase 2: Transformation Pipeline
        transformation ← new vtkTransform()
        reslice ← new vtkImageReslice()
        reslice.SetInputData(imageData)
        reslice.SetResliceTransform(transformation)
        transformed_data ← reslice.GetOutput()

    Phase 3: Visualization Paths
        Path A: 3D Volume Rendering
            volumeMapper.SetInputData(transformed_data)
            Setup transfer functions (color, opacity)
            volume ← new vtkVolume()
            renderer.AddVolume(volume)

        Path B: 2D Slice Extraction
            sliceMapper ← new vtkImageSliceMapper()
            sliceMapper.SetInputData(transformed_data)
            sliceMapper.SetOrientation(axial/coronal/sagittal)
            imageSlice ← new vtkImageSlice()
            renderer.AddViewProp(imageSlice)

    Phase 4: Interactive Rendering
        renderer.ResetCamera()
        window.Render()
        interactor.Initialize()
end procedure
```

Algorithm 4 Interactive Measurement System

CLASS *MeasurementInteractor* **EXTENDS** vtkInteractorStyleTrackballCamera
▷ Handles interactive measurements in 3D space

ATTRIBUTES:
    points: List[(x,y,z)] ▷ Measurement points
    mode: {Distance, Angle, Area, Volume}
    drawn_actors: List[vtkActor] ▷ Visual feedback

procedure ON_LEFT_CLICK(obj, event)
    click_pos ← GetInteractor().GetEventPosition()
    picker ← new vtkCellPicker()
    picker.SetTolerance(0.001)
    picker.Pick(click_pos[0], click_pos[1], 0, renderer)
    picked_pos ← picker.GetPickPosition()

    if picked_pos ≠ (0,0,0) **then**
        points.append(picked_pos)
        _mark_point(picked_pos) ▷ Add visual marker
        _process_measurement_mode()
    end if
end procedure

procedure _MEASURE_DISTANCE
Require: len(points) = 2
    p1, p2 ← points[0], points[1]
    dx, dy, dz ← p2 - p1
    distance ← $\sqrt{dx^2 + dy^2 + dz^2}$
    spacing ← imageData.GetSpacing()
    distance_mm ← distance × mean(spacing)
    _draw_line(p1, p2) ▷ Visual feedback
    _update_ui(f"Distance: {distance_mm:.2f} mm")
    points.clear()
end procedure

procedure _MEASURE_ANGLE
Require: len(points) = 3
    p1, p2, p3 ← points[0], points[1], points[2]
    v1 ← [p1[i] - p2[i] for i in 0..2]
    v2 ← [p3[i] - p2[i] for i in 0..2]
    dot ← $\sum_{i=0}^{2}$ v1[i] × v2[i]
    mag1 ← $\sqrt{\sum}$ v1[i]^2
    mag2 ← $\sqrt{\sum}$ v2[i]^2
    angle ← arccos(dot / (mag1 × mag2))
    angle_deg ← angle × 180/π
    _update_ui(f"Angle: {angle_deg:.2f}°")
    points.clear()
end procedure

Algorithm 5 Organ Segmentation System

Require: vtkImageData (DICOM volume), organ_type
Ensure: 3D surface model of segmented organ

```
procedure SEGMENT_ORGAN(min_hu, max_hu, organ_name)
    Step 1: Thresholding
    threshold ← new vtkImageThreshold()
    threshold.SetInputData(imageData)
    threshold.ThresholdBetween(min_hu, max_hu)
    threshold.SetInValue(1)                          ▷ Foreground
    threshold.SetOutValue(0)                         ▷ Background
    threshold.Update()

    Step 2: Connectivity Filtering
    connectivity ← new vtkImageConnectivityFilter()
    connectivity.SetInputConnection(threshold.GetOutputPort())
    connectivity.SetExtractionModeToLargestRegion()
    connectivity.Update()
    binary_mask ← connectivity.GetOutput()

    Step 3: Surface Generation
    marching ← new vtkMarchingCubes()
    marching.SetInputData(binary_mask)
    marching.SetValue(0, 0.5)                        ▷ Iso-surface value
    marching.ComputeNormalsOn()
    marching.Update()
    surface ← marching.GetOutput()

    Step 4: Visualization
    mapper ← new vtkPolyDataMapper()
    mapper.SetInputConnection(marching.GetOutputPort())
    mapper.ScalarVisibilityOff()

    actor ← new vtkActor()
    actor.SetMapper(mapper)
    actor.GetProperty().SetColor(0.9, 0.7, 0.3)      ▷ Gold color
    actor.GetProperty().SetOpacity(0.9)

    renderer.AddActor(actor)
    volume.SetVisibility(False)                      ▷ Hide main volume
    window.Render()
end procedure

                                  ▷ Organ HU Ranges (Hounsfield Units)
Bone: [200, 3000]
Lungs: [-1000, -600]
Liver: [50, 70]
Kidneys: [30, 50]
Soft Tissue: [-50, 100]
Muscle: [10, 40]
```

Algorithm 6 Inter-module Communication

Communication Pattern: Event-Driven with Signals/Slots

UI ↔ VTK Engine:
UI.Event → Controller.Method → VTK.Update → Render
VTK.RenderComplete → Callback → UI.StatusUpdate

Slice Viewer ↔ Main View Synchronization:
procedure SYNC_SLICES
    if slice_viewer.sync_enabled **then**
        slice_viewer.sliceChanged → main_view.set_slice
        main_view.orientationChanged → slice_viewer.set_orientation
    end if
end procedure

Measurement System Flow:
UserClick → Picker → 3D Coordinates
→ Geometry Calculation → Visual Feedback
→ UI Display → History Storage

Data Export Flow:
ExportRequest → Data Processor → Format Converter
→ File Writer → Status Notification

Algorithm 7 Rendering Optimization Strategies

Optimization Level 1: Adaptive Sampling
procedure SET_RENDERING_QUALITY(level) level 1 ▷ Very Low
    volumeMapper.SetSampleDistance(4.0) 2 ▷ Low
    volumeMapper.SetSampleDistance(2.0) 3 ▷ Medium
    volumeMapper.SetSampleDistance(1.0) 4 ▷ High
    volumeMapper.SetSampleDistance(0.5) 5 ▷ Very High
    volumeMapper.SetSampleDistance(0.25)
end procedure

Optimization Level 2: Surface Simplification
procedure OPTIMIZE_SURFACE(quality_level)
    decimate ← new vtkDecimatePro()
    reduction ← [0.9, 0.7, 0.4, 0.2, 0.0][quality_level-1]
    decimate.SetTargetReduction(reduction)
    decimate.PreserveTopologyOn()
end procedure

Memory Management:
- Lazy loading of DICOM slices
- vtkSmartPointer for automatic memory management
- Cache frequently used transformations
- Progressive rendering for large datasets

UI Responsiveness:
- Asynchronous data loading
- Debounced UI updates for sliders
- Separate rendering thread

STRUCT ApplicationState
▷ Encapsulates complete application state

Datasets:
    active_dataset: DatasetInfo
    loaded_datasets: Map¡id: DatasetInfo¿

Visualization Settings:
    view_mode: {3D, AxialSlice, CoronalSlice, SagittalSlice}
    render_mode: {Volume, Surface, MIP, MinIP}
    transfer_function: Preset or Custom
    window/level: (width, center)

Transformations:
    translation: (x, y, z) in mm
    rotation: (rx, ry, rz) in degrees
    scaling: factor

Measurements:
    active_measurements: List[Measurement]
    history: List[MeasurementResult]

Segmentation:
    segmentations: Map¡name: SegmentationMask¿
    active_segmentation: SegmentationMask

Camera State:
    position: (x, y, z)
    focal_point: (x, y, z)
    view_up: (x, y, z)
    clipping_range: (near, far)

Algorithm 8 Application State Representation

Algorithm 9 Multi-format Export Pipeline

PIPELINE ExportData(format, data_type)
format DICOM
writer ← new vtkDICOMWriter()
writer.SetInputData(imageData)
writer.SetDirectoryName(output_dir)
writer.SetFilePrefix("modified_")
writer.Write() PNG/JPEG/TIFF
window_to_image ← new vtkWindowToImageFilter()
window_to_image.SetInput(window)
writer ← format_specific_writer()
writer.SetFileName(output_file)
writer.Write() VTK Volume
writer ← new vtkXMLImageDataWriter() ▷ .vti
writer.SetInputData(imageData)
writer.SetFileName(output_file)
writer.Write() Complete Scene
multiBlock ← new vtkMultiBlockDataSet()
multiBlock.SetBlock(0, imageData)
multiBlock.SetBlock(1, settings_json)
writer ← new vtkXMLMultiBlockDataWriter()
writer.Write() ▷ .vtm file
=0

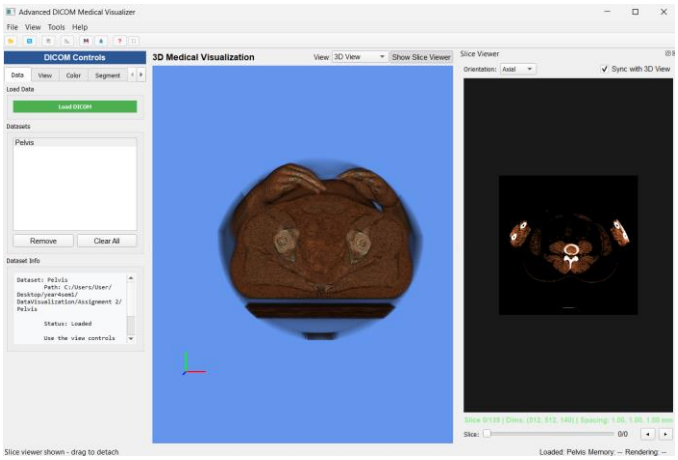
Pattern	Implementation	Purpose
MVC	AdvancedDICOMViewer (Controller)	Separation of concerns
Observer	PyQt5 Signals/Slots	Event-driven updates
Strategy	Rendering modes, Measurement algorithms	Algorithm interchangeability
Composite	Multi-block scene organization	Hierarchical data management
Factory	Dataset loaders, Export writers	Object creation abstraction
Singleton	Application instance	Single point of access

Algorithm 10 Application Startup Sequence

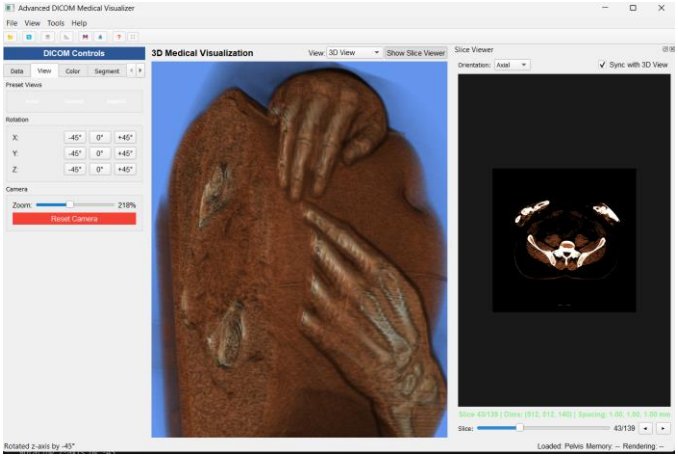
```
1: procedure MAIN
2:     app ← new QApplication(sys.argv)
3:     app.setStyle('Fusion')
4:
5:     window ← new AdvancedDICOMViewer()
6:     window.show()
7:
8:                                                    ▷ Setup global shortcuts
9:     fullscreen_shortcut ← new QShortcut("F11", window)
10:    fullscreen_shortcut.activated.connect(window.toggle_fullscreen)
11:
12:    escape_shortcut ← new QShortcut("Escape", window)
13:    escape_shortcut.activated.connect(exit_fullscreen_handler)
14:
15:    sys.exit(app.exec_())
16: end procedure
```

Output

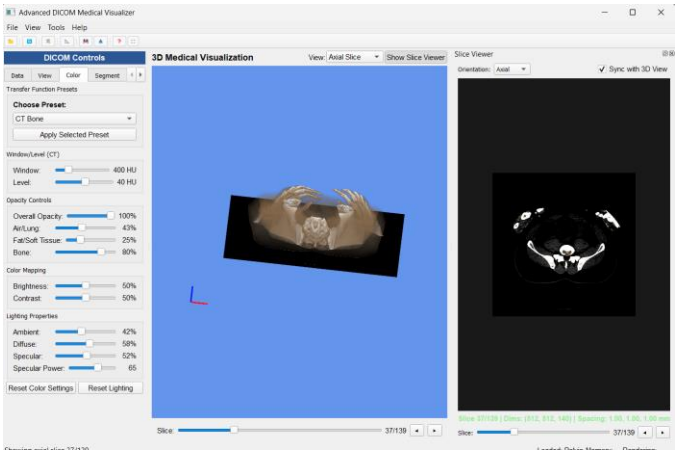
Data Tab



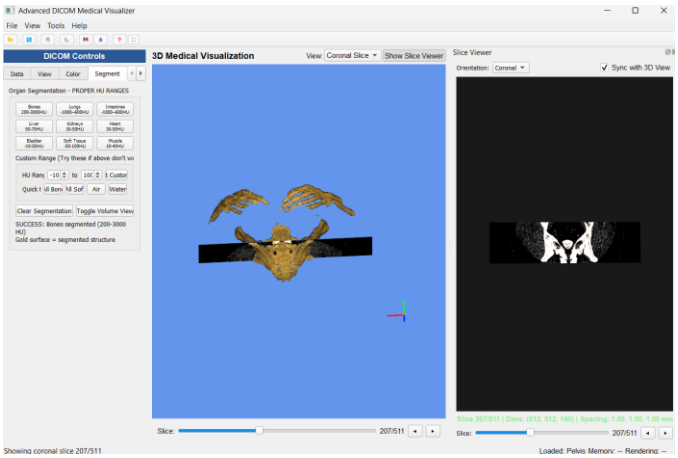
View Tab



Color Tab

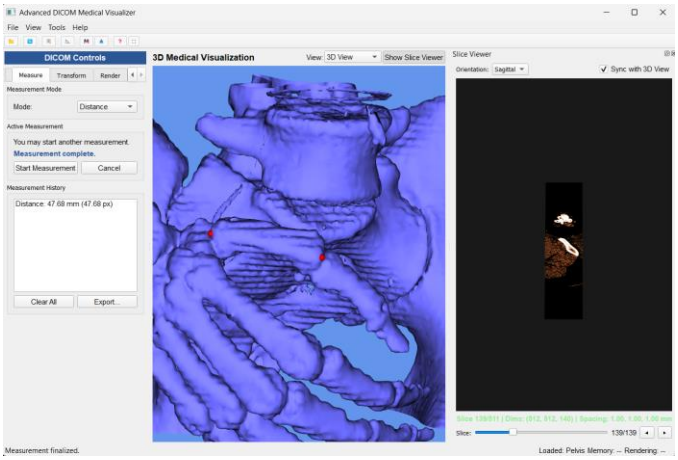


Segment Tab

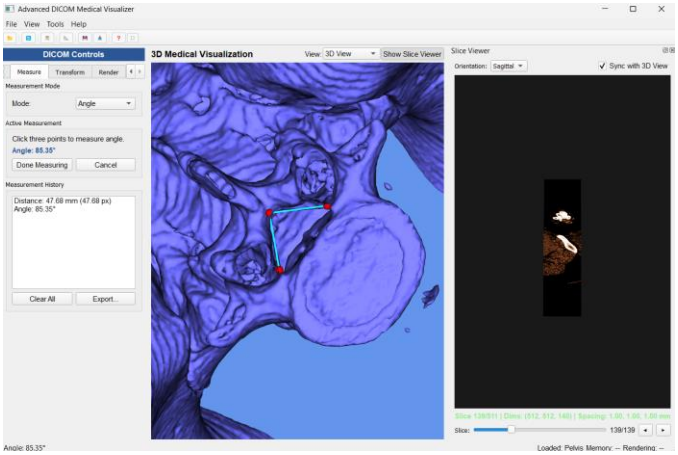


Measure Tab

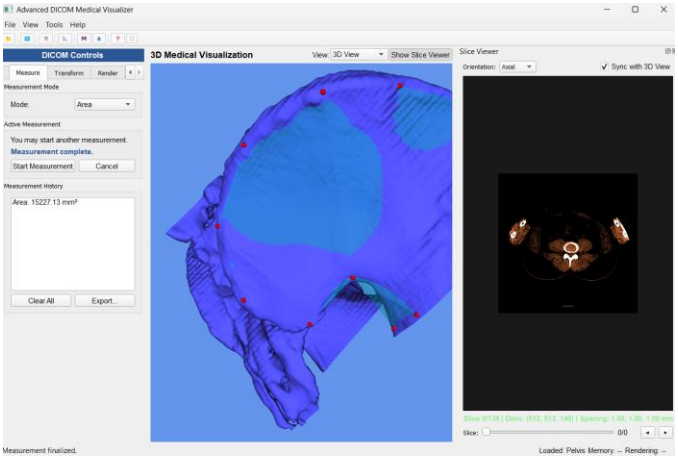
Measuring Distance



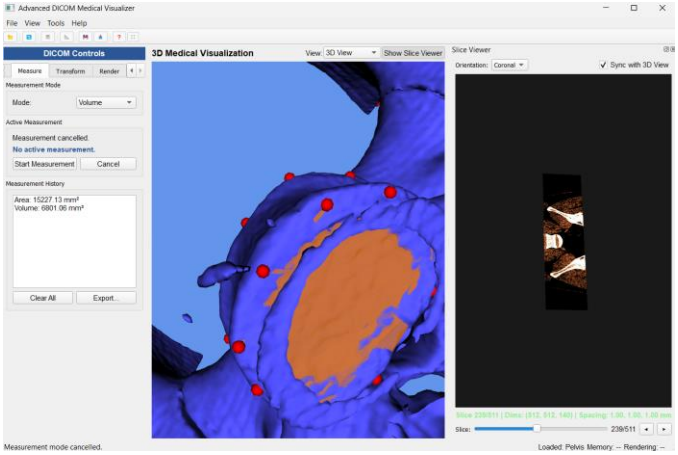
Measuring Angle



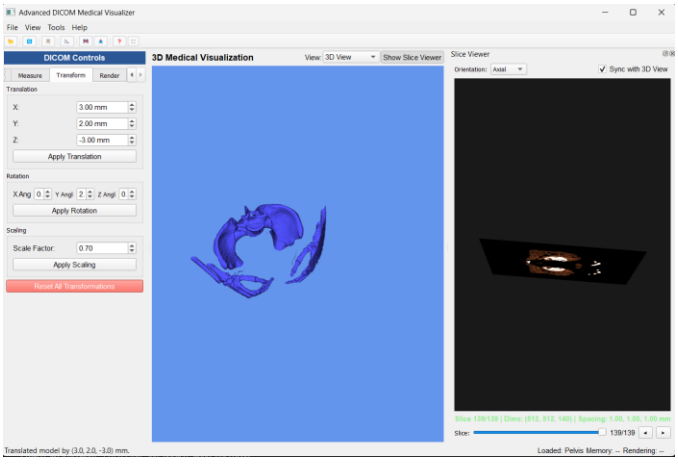
Measuring Area



Measuring Volume

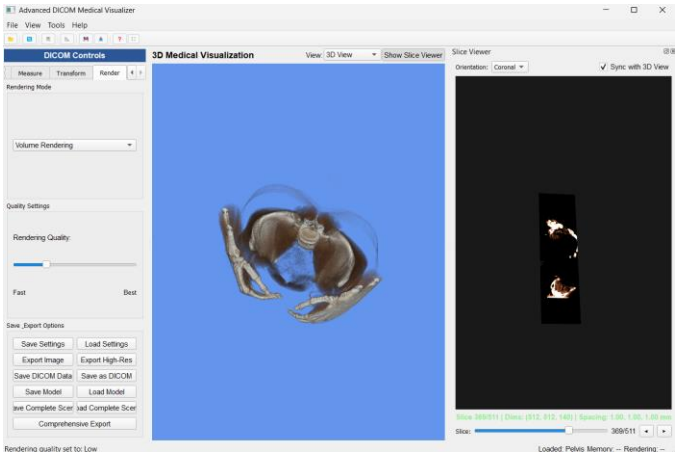


Transform Tab

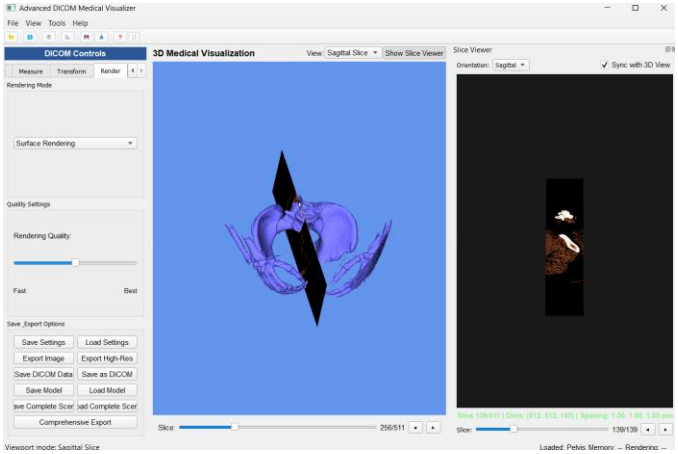


Render Tab

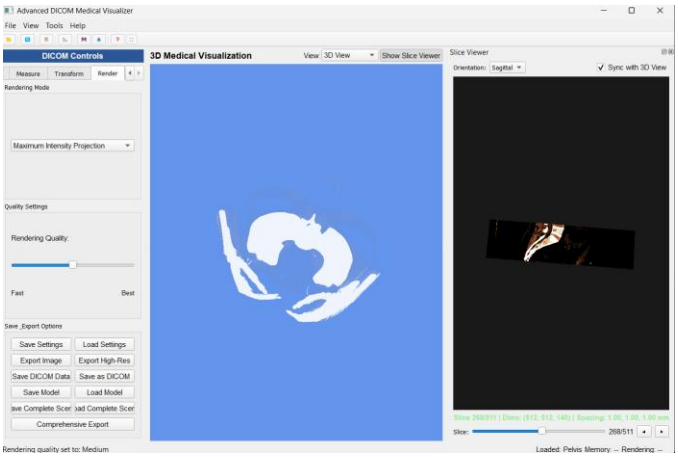
Volume Rendering



Surface Rendering



Maximum Intensity Projection



Minimum Intensity Projection